

Lab 06 – Comparing UDP and TCP in Wireshark

Name: Mitchell Cuvar

Course/Section: IS-3413

Date: March 25, 2026

INTRODUCTION

This lab focuses on analyzing and comparing the behavior of UDP and TCP protocols using Wireshark. The objective is to understand how each protocol operates at the transport layer, including their structure, reliability mechanisms, and use cases. By examining captured traffic, this lab highlights key differences between connection-oriented and connectionless communication.

BREAKPOINT 1

In this step, I prepared my Wireshark environment by downloading the provided pcap file and loading it into Wireshark. I accessed the file through the provided lab resources and opened it using the “File → Open” option in Wireshark. Once loaded, I ensured that the packet capture displayed properly and adjusted the layout if necessary to clearly view packet details, protocol hierarchy, and header information.

Below is the pcap file I opened in Wireshark, demonstrating all the necessary fields.

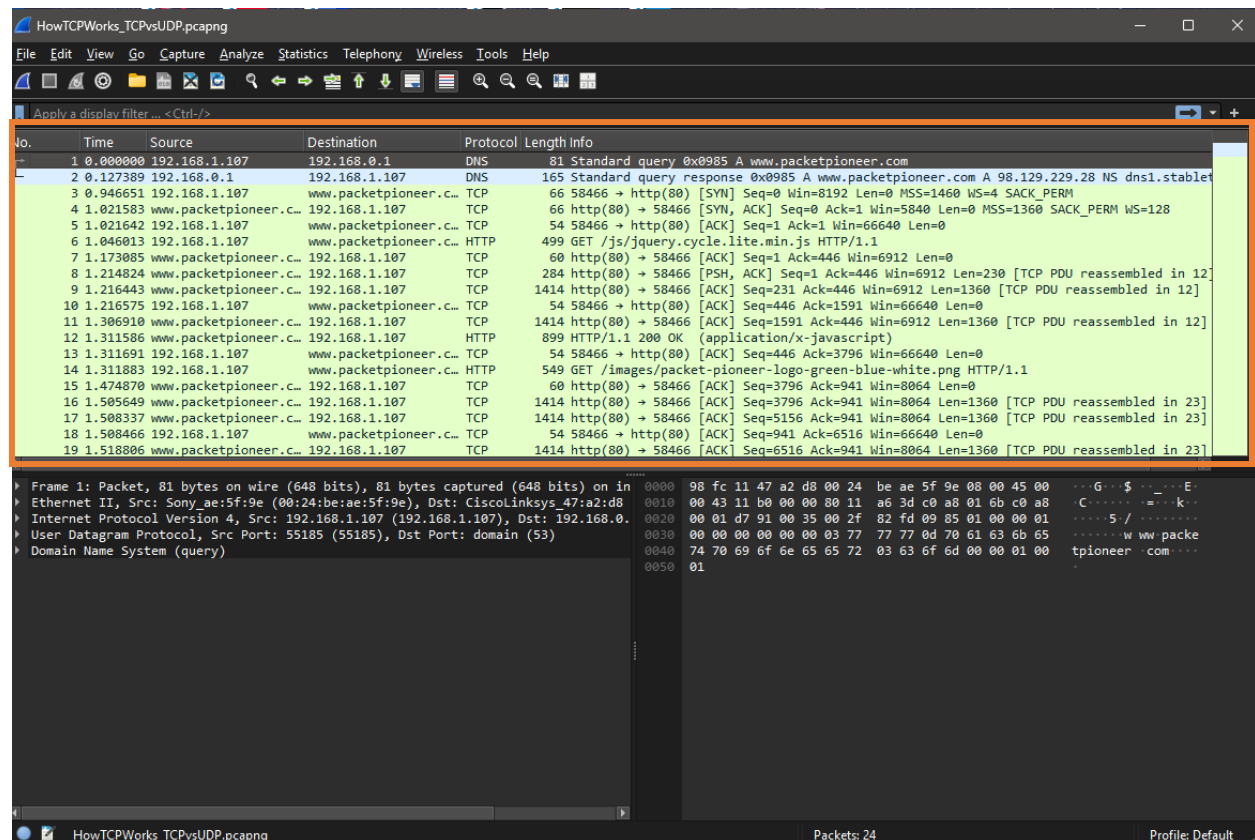


Figure 1: Wireshark interface with the pcap file

BREAKPOINT 2

UDP and TCP differ primarily in how they handle communication speed and reliability. TCP is a connection-oriented protocol, meaning it establishes a connection between devices before transmitting data. It ensures reliable delivery through acknowledgments, retransmissions, and sequencing. UDP, on the other hand, is connectionless and does not guarantee delivery, ordering, or duplication protection.

TCP is considered stateful because it maintains information about the connection, such as sequence numbers and acknowledgment states. This allows it to track data transmission and ensure accuracy. UDP is stateless, meaning it sends packets without maintaining any session information or tracking whether they arrive successfully.

Because TCP prioritizes reliability, it introduces overhead and latency due to its handshake process and error-checking mechanisms. UDP sacrifices reliability for speed and efficiency, making it ideal for applications where real-time performance is more important than perfect accuracy.

BREAKPOINT 3

The protocol running over UDP in this capture is DNS (Domain Name System). This is evident in the Wireshark packet list where UDP packets are associated with DNS queries and responses.

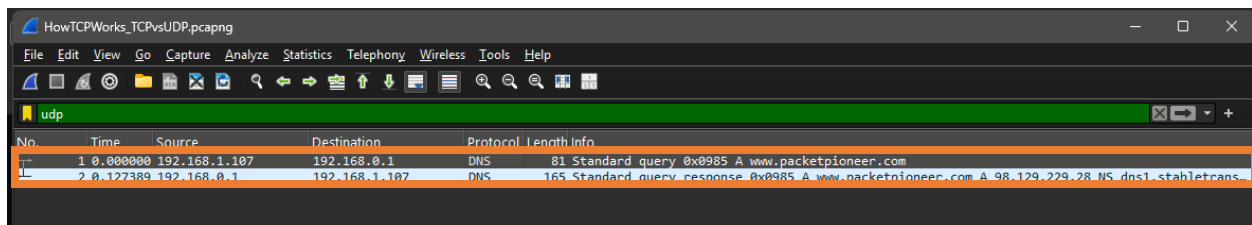


Figure 2: UDP packets identified as DNS

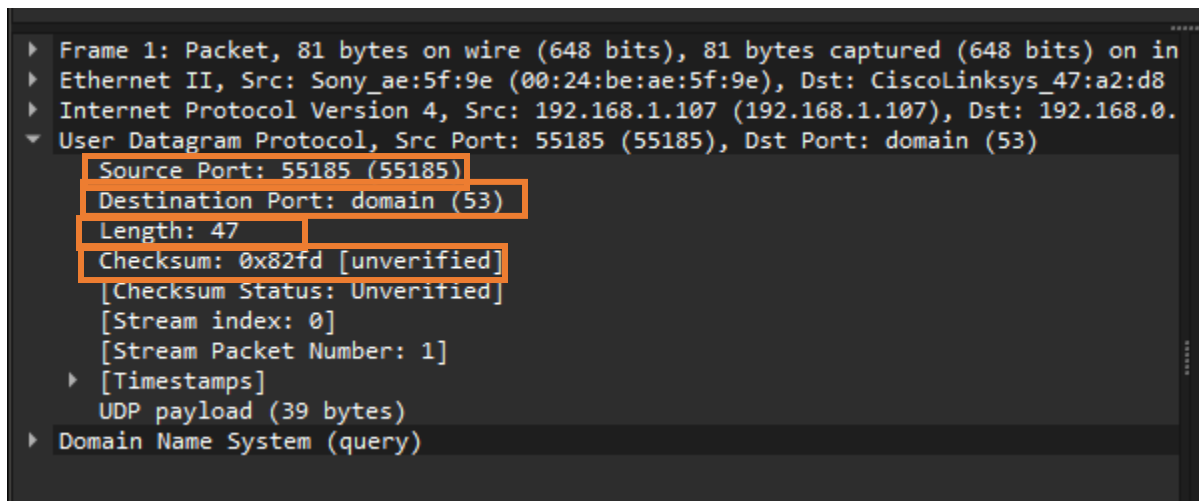


Figure 3:UDP header fields

The UDP header contains four main fields:

- **Source Port:** Identifies the sending application.
- **Destination Port:** Identifies the receiving application.
- **Length:** Specifies the total length of the UDP segment.
- **Checksum:** Used for error detection.

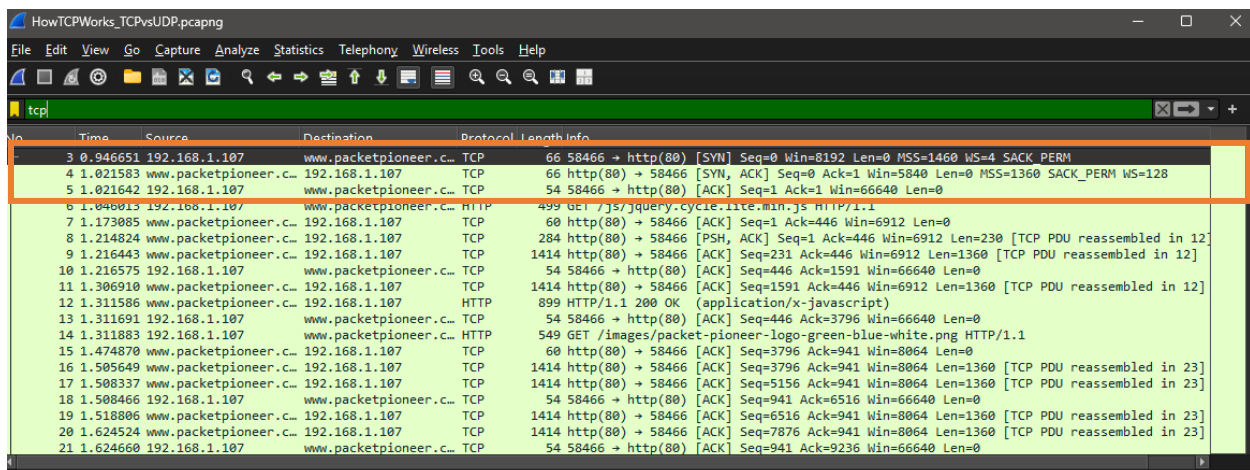
Each of these fields plays a role in delivering data quickly with minimal overhead. Unlike TCP, UDP does not include sequencing or acknowledgment fields, which explains its simplicity and speed.

BREAKPOINT 4

The TCP three-way handshake consists of:

1. **SYN:** The client initiates a connection by sending a synchronization request.
2. **SYN-ACK:** The server acknowledges the request and responds with its own synchronization.
3. **ACK:** The client acknowledges the server, completing the connection.

This handshake establishes a reliable connection before any data is transmitted.



The image shows a Wireshark packet capture window titled 'HowTCPWorks_TCPvsUDP.pcapng'. The filter is set to 'tcp'. The packet list shows the following packets:

No.	Time	Source	Destination	Protocol	Length	Info
3	0.946651	192.168.1.107	www.packetpioneer.com	TCP	66	58466 → http(80) [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM
4	1.021583	www.packetpioneer.com	192.168.1.107	TCP	66	http(80) → 58466 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1360 SACK_PERM WS=128
5	1.021642	192.168.1.107	www.packetpioneer.com	TCP	54	58466 → http(80) [ACK] Seq=1 Ack=1 Win=66640 Len=0
6	1.046015	192.168.1.107	www.packetpioneer.com	HTTP	499	GET /js/jquery.cycle-lite.min.js HTTP/1.1
7	1.173085	www.packetpioneer.com	192.168.1.107	TCP	60	http(80) → 58466 [ACK] Seq=1 Ack=446 Win=6912 Len=0
8	1.214824	www.packetpioneer.com	192.168.1.107	TCP	284	http(80) → 58466 [PSH, ACK] Seq=1 Ack=446 Win=6912 Len=230 [TCP PDU reassembled in 12]
9	1.216443	www.packetpioneer.com	192.168.1.107	TCP	1414	http(80) → 58466 [ACK] Seq=231 Ack=446 Win=6912 Len=1360 [TCP PDU reassembled in 12]
10	1.216575	192.168.1.107	www.packetpioneer.com	TCP	54	58466 → http(80) [ACK] Seq=446 Ack=1591 Win=66640 Len=0
11	1.306910	www.packetpioneer.com	192.168.1.107	TCP	1414	http(80) → 58466 [ACK] Seq=1591 Ack=446 Win=6912 Len=1360 [TCP PDU reassembled in 12]
12	1.311586	www.packetpioneer.com	192.168.1.107	HTTP	899	HTTP/1.1 200 OK (application/x-javascript)
13	1.311691	192.168.1.107	www.packetpioneer.com	TCP	54	58466 → http(80) [ACK] Seq=446 Ack=3796 Win=66640 Len=0
14	1.311883	192.168.1.107	www.packetpioneer.com	HTTP	549	GET /images/packet-pioneer-logo-green-blue-white.png HTTP/1.1
15	1.474870	www.packetpioneer.com	192.168.1.107	TCP	60	http(80) → 58466 [ACK] Seq=3796 Ack=941 Win=8064 Len=0
16	1.505649	www.packetpioneer.com	192.168.1.107	TCP	1414	http(80) → 58466 [ACK] Seq=3796 Ack=941 Win=8064 Len=1360 [TCP PDU reassembled in 23]
17	1.508337	www.packetpioneer.com	192.168.1.107	TCP	1414	http(80) → 58466 [ACK] Seq=5156 Ack=941 Win=8064 Len=1360 [TCP PDU reassembled in 23]
18	1.508466	192.168.1.107	www.packetpioneer.com	TCP	54	58466 → http(80) [ACK] Seq=941 Ack=6516 Win=66640 Len=0
19	1.518806	www.packetpioneer.com	192.168.1.107	TCP	1414	http(80) → 58466 [ACK] Seq=6516 Ack=941 Win=8064 Len=1360 [TCP PDU reassembled in 23]
20	1.624524	www.packetpioneer.com	192.168.1.107	TCP	1414	http(80) → 58466 [ACK] Seq=7876 Ack=941 Win=8064 Len=1360 [TCP PDU reassembled in 23]
21	1.624660	192.168.1.107	www.packetpioneer.com	TCP	54	58466 → http(80) [ACK] Seq=941 Ack=9236 Win=66640 Len=0

Figure 4: TCP three-way handshake:

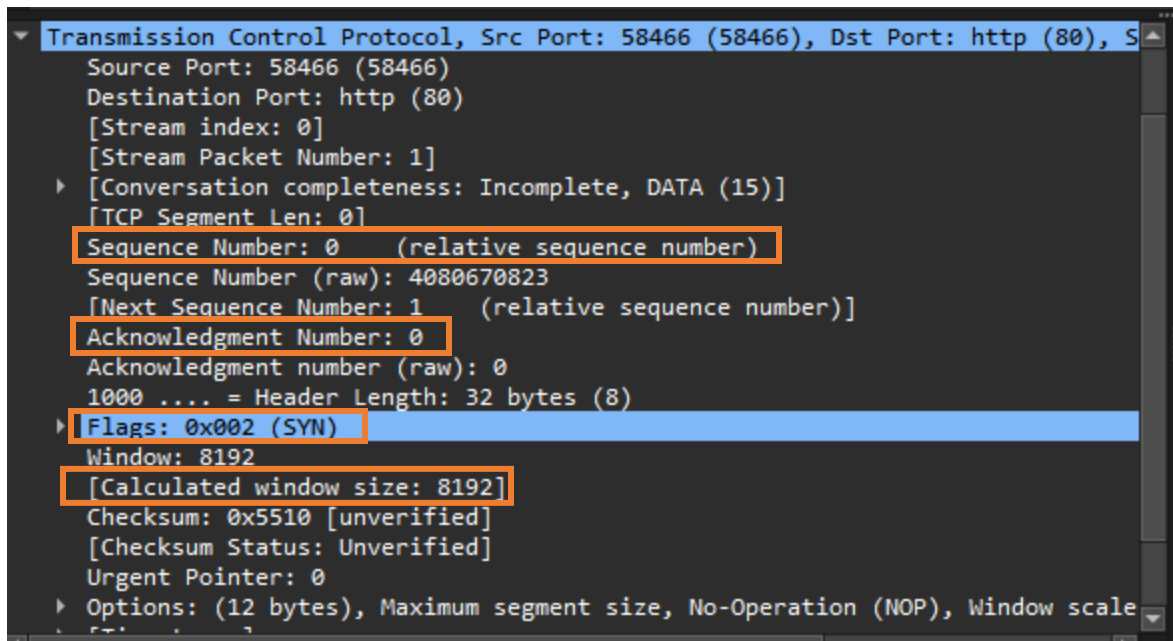


Figure 5: TCP header fields

The TCP header includes several fields not found in UDP:

- **Sequence Number:** Tracks the order of data.
- **Acknowledgment Number:** Confirms receipt of data.
- **Flags (SYN, ACK, FIN, etc.):** Control connection state.
- **Window Size:** Manages flow control.
- **Checksum:** Ensures data integrity.
- **Urgent Pointer (optional):** Indicates urgent data.

These additional fields enable TCP to provide reliable, ordered communication, which is why it is more complex than UDP.

BREAKPOINT 5

TCP controls congestion through mechanisms such as flow control, congestion avoidance, and retransmission strategies. It dynamically adjusts the rate of data transmission based on network conditions, ensuring that the network is not overwhelmed.

UDP is preferred in situations where low latency is critical, such as video streaming, online gaming, and VoIP. TCP is preferred when reliability is essential, such as file transfers, web browsing, and email.

An interesting feature observed in the pcap is how TCP packets show a clear sequence of acknowledgments, demonstrating reliable communication, while UDP packets appear more sporadic and lack confirmation responses. This highlights the fundamental design difference between the two protocols.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.107	192.168.0.1	DNS	81	Standard query 0x0985 A www.packetpioneer.com
2	0.127389	192.168.0.1	192.168.1.107	DNS	165	Standard query response 0x0985 A www.packetpioneer.com A 98.129.229.28 NS dns1.stable
3	0.946651	192.168.1.107	www.packetpioneer.c...	TCP	66	58466 → http(80) [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM
4	1.031583	www.packetpioneer.c...	192.168.1.107	TCP	66	http(80) → 58466 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1360 SACK_PERM WS=128
5	1.021642	192.168.1.107	www.packetpioneer.c...	TCP	54	58466 → http(80) [ACK] Seq=1 Ack=1 Win=66640 Len=0
6	1.046013	192.168.1.107	www.packetpioneer.c...	HTTP	499	GET /js/jquery.cycle.lite.min.js HTTP/1.1
7	1.173085	www.packetpioneer.c...	192.168.1.107	TCP	60	http(80) → 58466 [ACK] Seq=1 Ack=446 Win=6912 Len=0
8	1.214824	www.packetpioneer.c...	192.168.1.107	TCP	284	http(80) → 58466 [PSH, ACK] Seq=1 Ack=446 Win=6912 Len=230 [TCP PDU reassembled in 12]
9	1.216443	www.packetpioneer.c...	192.168.1.107	TCP	1414	http(80) → 58466 [ACK] Seq=231 Ack=446 Win=6912 Len=1360 [TCP PDU reassembled in 12]
10	1.216575	192.168.1.107	www.packetpioneer.c...	TCP	54	58466 → http(80) [ACK] Seq=446 Ack=1591 Win=66640 Len=0
11	1.306910	www.packetpioneer.c...	192.168.1.107	TCP	1414	http(80) → 58466 [ACK] Seq=1591 Ack=446 Win=6912 Len=1360 [TCP PDU reassembled in 12]
12	1.311586	www.packetpioneer.c...	192.168.1.107	HTTP	899	HTTP/1.1 200 OK (application/x-javascript)
13	1.311691	192.168.1.107	www.packetpioneer.c...	TCP	54	58466 → http(80) [ACK] Seq=446 Ack=3796 Win=66640 Len=0
14	1.311883	192.168.1.107	www.packetpioneer.c...	HTTP	549	GET /images/packet-pioneer-logo-green-blue-white.png HTTP/1.1
15	1.474870	www.packetpioneer.c...	192.168.1.107	TCP	60	http(80) → 58466 [ACK] Seq=3796 Ack=941 Win=8064 Len=0
16	1.505649	www.packetpioneer.c...	192.168.1.107	TCP	1414	http(80) → 58466 [ACK] Seq=3796 Ack=941 Win=8064 Len=1360 [TCP PDU reassembled in 23]
17	1.508337	www.packetpioneer.c...	192.168.1.107	TCP	1414	http(80) → 58466 [ACK] Seq=5156 Ack=941 Win=8064 Len=1360 [TCP PDU reassembled in 23]
18	1.508466	192.168.1.107	www.packetpioneer.c...	TCP	54	58466 → http(80) [ACK] Seq=941 Ack=6516 Win=66640 Len=0
19	1.518806	www.packetpioneer.c...	192.168.1.107	TCP	1414	http(80) → 58466 [ACK] Seq=6516 Ack=941 Win=8064 Len=1360 [TCP PDU reassembled in 23]

Figure 6: Comparison of TCP(orange) and UDP(red)

CONCLUSION

Replace the text in this section with a reflective summary of your process in this lab, including answers to the following, in complete sentences:

- What challenges did you encounter and/or what most interested you about this lab?

One challenge in this lab was understanding how to interpret the different fields in Wireshark and relate them to theoretical concepts from the textbook. However, by reviewing the protocol structures and observing real packet data, I was able to better understand how TCP and UDP function in practice.

- How did you address your challenges and/or what additional research did you do?

Additional research and reviewing the video helped clarify how connection-oriented communication works and why TCP includes more overhead. This lab reinforced the importance of choosing the correct protocol based on application needs.

- How might you use the skills and concepts of this lab going forward?

These concepts will be useful in future networking tasks, especially when analyzing traffic, troubleshooting network issues, or designing systems that require specific performance characteristics.

REFERENCES

R. Mitra, "Lab 06 – Comparing UDP and TCP in Wireshark," The University of Texas at San Antonio (2025). Last accessed: March 25, 2026.

[1] C. Greer, "TCP vs UDP Explained // Hands On Lab Example with Wireshark," YouTube. Accessed: March 25, 2026.

GENERATIVE AI SEARCHES

CHATGPT [GPT-5.3], RESPONSE TO “SUMMARIZE THE DIFFERENCES BETWEEN UDP AND TCP AND EXPLAIN WIRESHARK LAB QUESTIONS,” OPENAI, 2026. ACCESSED: MARCH 25, 2026.

COLLABORATION

I worked independently